

VERIQO Unified Verifier Specification

The specification for verifying a VERIQO Machine Package. It unifies what was previously described in two places (the insurance-domain dossier verifier and the Commercial package verifier) into one statement of what is checked, what each result means, and — critically — what a `SKIP` does and does not tell you.

Implementations: `pkg/commercial/packageverify` (the single implementation of every check) and `cmd/veriqo-commercial-verify` (the standalone binary). The HTTP route `POST /v1/packages/verify` calls the *same* package in-process, so there is exactly one implementation of these checks and never two that could drift apart.

1. Why a separate binary exists

The verifier is a genuinely separate OS process that takes only a `.zip` file from disk and, optionally, a trusted-key registry the verifying party obtained through their own channel. It makes no network calls, imports no running service, and shares no memory with any VERIQO deployment.

The commercial proposition is exactly this: **"Don't trust VERIQO. Verify it yourself."** A verifier that had to ask a VERIQO server whether a package was valid would be worthless for the disputes this product exists to serve — the counterparty is precisely the party who does not want to take your server's word for anything.

2. Usage

```
veriqo-commercial-verify -package path/to/case.zip [-trusted-keys keys.json]
```

Exit codes

Code	Meaning
0	ALL CHECKS PASSED. <code>SKIP</code> lines are reported but are not failures.
1	One or more checks FAILED.
2	Usage or input error (missing file, unreadable zip, malformed key registry).

Trusted-key registry format — a JSON object keyed by key ID:

```
{
  "veriqo-evidence-key-1": {
    "public_key": "<hex-encoded Ed25519 public key>",
    "revoked": false
  }
}
```

Omitting `-trusted-keys` is supported and honest: signature and key-state checks then report `SKIP` with the reason, never a false `PASS`.

3. The three statuses

Status	Meaning
PASS	The check was performed and the property holds.

Status	Meaning
FAIL	The check was performed and the property does not hold. The package is not sound.
SKIP	The check could not be independently performed with the input supplied. It is neither a pass nor a failure.

AllPassed — and therefore exit code 0 — treats `SKIP` as not-a-failure. **This is the single most important thing to understand about a verification report.** A run that exits 0 with signature checks skipped has proven structural and hash integrity, and has proven *nothing at all* about who signed it.

A reader evaluating a verification report must read the `SKIP` lines, not just the verdict.

4. The checks

Checks are emitted per-artifact where relevant (evidence-scoped checks are suffixed with the evidence ID; key-state checks with the key ID).

Package-level

Check	Verifies
<code>package_hash</code>	The dossier's own package hash recomputes over its content — with both the hash field and the signature field zeroed, so a signature is never part of what it signs.
<code>package_signature</code>	Real Ed25519 verification of the dossier signature against the trusted registry. <code>SKIP</code> when unsigned or when no registry was supplied.
<code>manifest_data_present</code>	The package actually contains the manifest data it claims to.

Per evidence item

Check	Verifies
<code>manifest[<id>]</code>	The evidence manifest hash recomputes over its canonicalized content — the record is unaltered.
<code>raw_evidence_hash[<id>]</code>	A SHA-256 is recorded for the item. <code>FAIL</code> when a manifest carries none.
<code>custody_chain[<id>]</code>	The custody events form an unbroken hash chain from GENESIS; the detail line reports how many events were verified.
<code>signature[<id>]</code>	Real Ed25519 verification of the evidence signature. <code>SKIP</code> when unsigned or no registry.

Key state

Check	Verifies
<code>key_state[<key id>]</code>	The signing key is present in the supplied trusted registry and is not revoked . <code>FAIL</code> when the registry marks it revoked — so a signature made by a key that was later revoked fails verification retroactively. <code>SKIP</code> when no registry entry exists for that key.

Ledger

Check	Verifies
<code>ledger_hash_chain</code>	Every ledger record is replayed from GENESIS and each hash independently re-derived and matched. Detects any insertion, deletion, reordering, or edit.
<code>merkle_root</code>	The Merkle root over the ledger records is computable and reported, giving a single value to compare against an external anchor.

Check	Verifies
<code>lineage_decision</code>	The Decision hash the dossier claims actually appears in the independently-parsed ledger.
<code>lineage_authorization</code>	The same for the Action authorization hash, when the dossier carries one.

5. Why the lineage checks matter

`package_hash` proves the dossier is internally consistent. It does not prove the dossier is *about* anything real: a sufficiently determined forger could produce an internally consistent dossier claiming a Decision that never happened, recomputing its own package hash to match.

The lineage checks close that: the dossier's claimed Decision and Authorization hashes are cross-referenced against the ledger the verifier parsed *independently* from the same package. A forged dossier claiming a decision the ledger never recorded fails `lineage_decision` even though `package_hash` passes.

This is why the verifier parses the ledger itself rather than trusting the dossier's summary of it.

6. Scope — what the verifier does NOT do

Stated explicitly, because a verifier's honesty depends on its limits being legible:

- **It does not perform full input-level replay.** A Machine Package deliberately excludes the original Decision and Action *inputs*, so the verifier cannot re-derive a Decision from scratch. It verifies lineage and chain integrity. Full re-derivation is `Store.Replay`'s job against the live Store — see the Replay Specification, §5.
- **It does not check the evidence bytes.** It verifies that a SHA-256 is recorded and that the record around it is intact. Confirming the document you hold matches that hash is a step you perform yourself (`sha256sum yourfile.pdf` and compare) — VERIQO never had the bytes.
- **It does not establish authenticity without a trusted key registry.** See §7.
- **It does not validate external identities.** That a vessel identity, policy number, or party ID corresponds to a real entity in an authoritative registry is outside its scope and not built.
- **It takes no legal position.** See the Legal Positioning Statement.

7. The trust anchor

A Machine Package embeds the `key_id` that signed it. **That proves nothing on its own** — a forger controls what key ID they write into a package they fabricated.

Real signature verification therefore requires a trusted key registry obtained **outside the package**: a published registry, a key exchanged during onboarding, or a registry your organization maintains. The verifier deliberately does not accept the package's own self-declared key material as a trust anchor, because a self-referential trust model is not a trust model.

Consequence for deployment: if cryptographic authenticity matters to your counterparties, establishing key distribution is a real operational step you must plan. Without it, verification is structural and hash-based only — which is genuinely valuable, and is not authenticity.

8. The HTTP route's honest limitation

`POST /v1/packages/verify` runs the same checks in-process, but always with **no** trusted key registry — the raw-body upload shape has no way to carry one. It therefore always reports `SKIP` for signature and key-state checks.

It is a convenience for the package's own holder, not the verifier of record. Anyone who needs to verify a package *without trusting the server that produced it* must use the standalone CLI. Using the API route of the very party whose claims are in dispute would defeat the purpose.

9. Reading a report

```
[PASS] package_hash           recomputed over canonical content
[SKIP] package_signature      no trusted key registry supplied
[PASS] manifest[EV-1]        manifest hash re-derived and matches
[PASS] raw_evidence_hash[EV-1] sha256=a1b2...
[PASS] custody_chain[EV-1]    4 events, hash-chain verified from GENESIS
[SKIP] signature[EV-1]       no trusted key registry supplied
[PASS] ledger_hash_chain      12 records replayed from GENESIS, every hash
                              independently re-derived and matches

[PASS] merkle_root            root=4dd79a...
[PASS] lineage_decision       dossier's recorded hash matches a real ledger record
[PASS] lineage_authorization   dossier's recorded hash matches a real ledger record

VERDICT: ALL CHECKS PASSED (see SKIP lines above for checks this
reference build could not independently verify)
```

The correct reading of that report: *the record is internally intact, unaltered, and genuinely linked to the ledger it claims — and nothing here establishes which key produced it, because no trusted registry was supplied.* Both halves belong in any summary of the result.